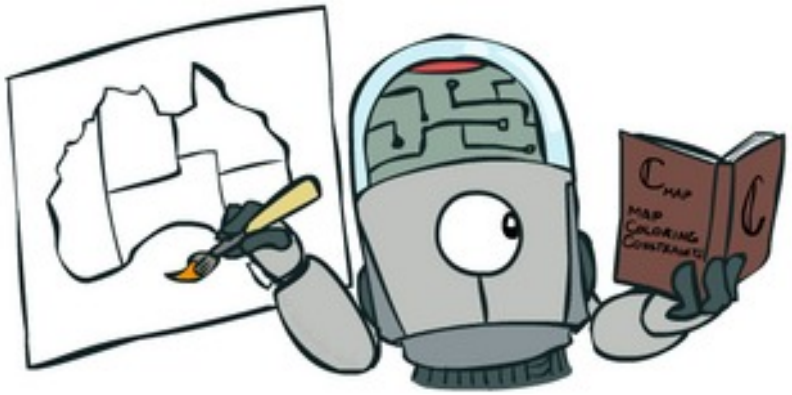
KU LEUVEN

Fundamentals of Artificial Intelligence



Prof. Tias Guns

Lecture 05, 2026-27
Constraint Satisfaction Problems

Based on the slides of CS188 @ Berkeley

FAI: Reasoning & decisions

L01:	Mon 21/09, 10:35 -- 12:30,	Intro AI
L02:	Mon 28/09, 10:35 -- 12:30,	Agents + Search (1/2)
L03:	Mon 05/10, 10:35 -- 12:30,	Search (2/2)
HS1:	Mon 05/10, **17:00 -- 18:30**,	#Project hackerspace 1: get rollin'
L04:	Mon 12/10, 10:35 -- 12:30,	Informed Search
L05:	**Mon 19/10, 10:35 -- 12:30,**	**Constraint Satisfaction Problems**
L06:	Mon 26/10, 10:35 -- 12:30,	Game trees
HS2:	Fri 06/11, 12:35 -- 14:00,	#Project hackerspace 2: last resort
L07:	Mon 09/11, 10:35 -- 12:30,	Uncertainty in Game Trees
L08:	Mon 16/11, 10:35 -- 12:30,	Markov Decision Processes
L09:	Mon 23/11, 10:35 -- 12:30,	SAT Solving + Logic
L10:	Mon 30/11, 10:35 -- 12:30,	1st order Logic + Planning
LXX:	Mon 07/12, 10:35 -- 12:30,	Exam info + Q&A
LGG:	Thu 17/12, 13:00 – 16:00,	Guest Lecture: Ethics of AI

Algorithmic fundamentals of search

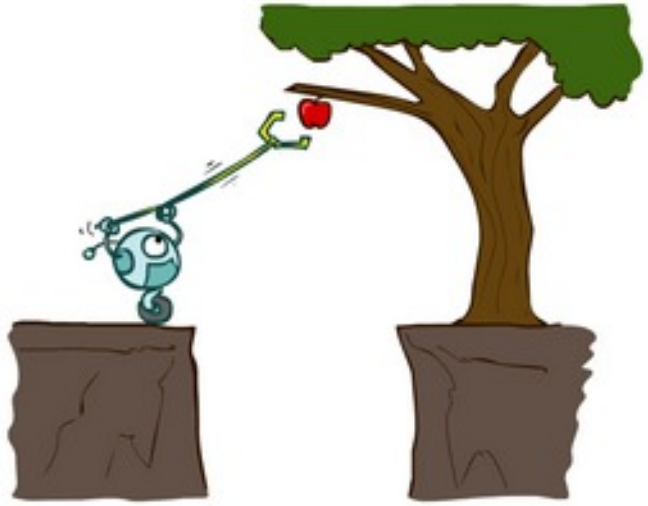
Applications of search in deterministic environments

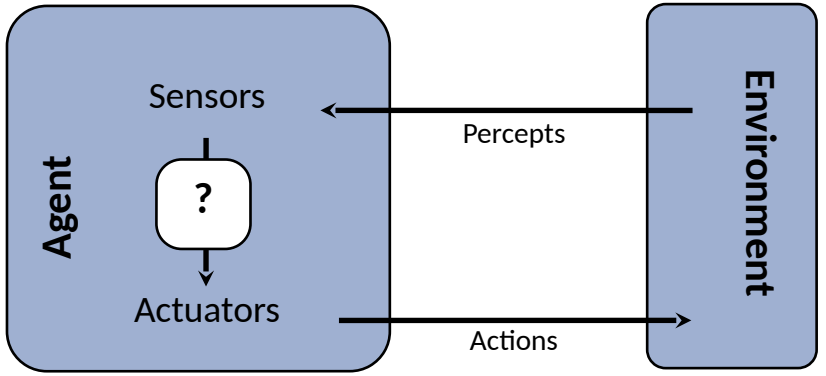
Applications of search in stochastic env.

Implications of AI

RECAP: Designing Rational Agents

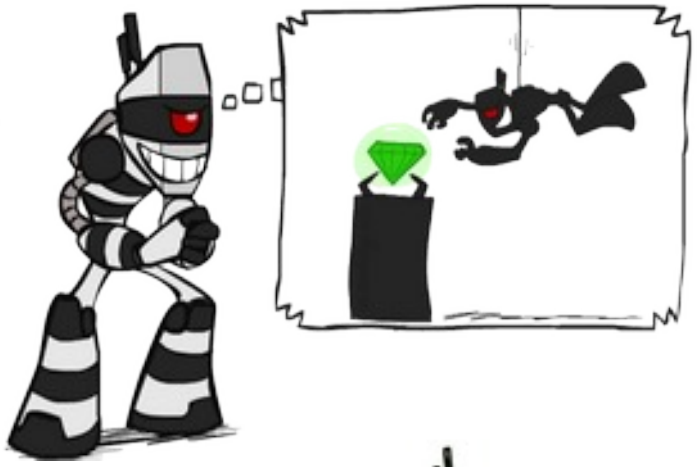
- An **agent** is an entity that *perceives* and *acts*.
- A **rational agent** selects actions that maximize its (expected) **utility**.
- Characteristics of the **percepts, environment,** and **action space** dictate techniques for selecting rational actions
- **This course** is about:
  - General AI **search** techniques for a variety of problem types
  - Learning to recognize when and how a new problem can be solved with an existing technique





What is Search For?

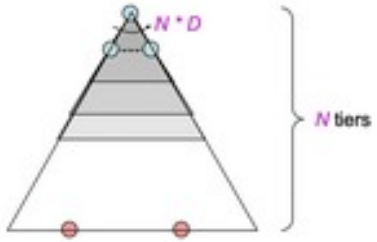
- Assumptions about the world: a single agent, deterministic actions, fully observed state, discrete state space



- Planning: sequences of actions
  - The path to the goal is the important thing
  - Paths have various costs, depths
  - Heuristics give problem-specific guidance

- Identification: assignments to variables
  - The goal itself is important, not the path
  - All paths at the same depth (for some formulations)





CSP Examples



https://mybinder.org/v2/gh/CPMpy/cpmpy/HEAD?labpath=examples%2Fmap_coloring_oz.ipynb

Example: Map Coloring

- Variables: WA, NT, Q, NSW, V, SA, T
- Domains: D = {red, green, blue}
- Constraints: adjacent regions must have different colors

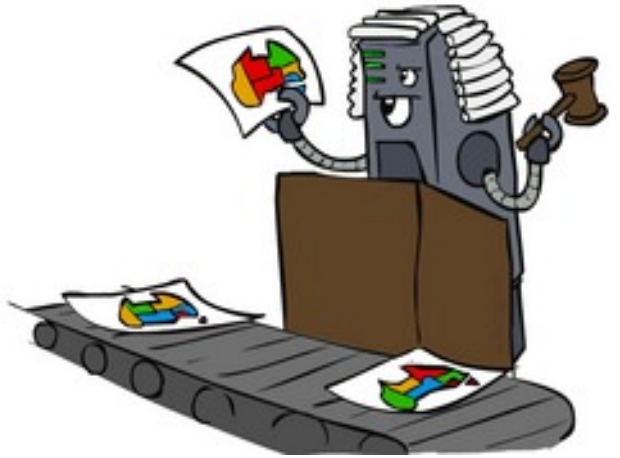
  Implicit representation: $WA \neq NT$

  Explicit representation: $(WA, NT) \in \{(red, green), (red, blue), \ldots\}$



- Solutions are assignments satisfying all constraints, e.g.:

  {WA=red, NT=green, Q=red, NSW=green, V=red, SA=blue, T=green}

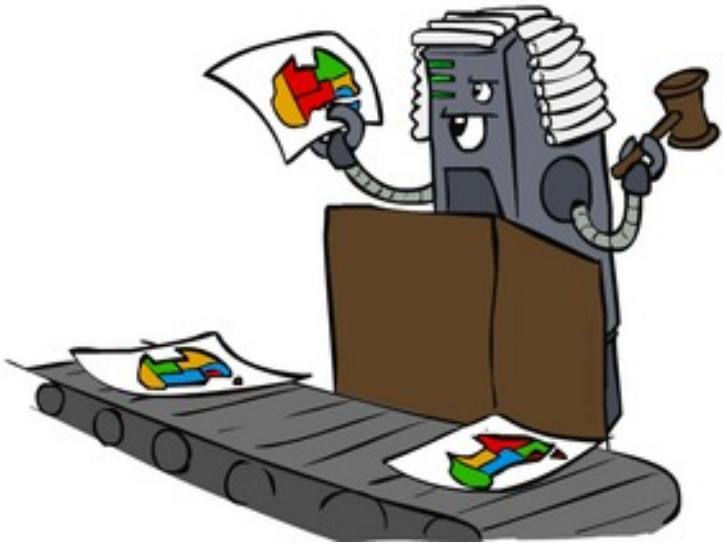


Constraint Satisfaction Problems

- Generic search problems:
  - State is a "black box": arbitrary data structure
  - Goal test can be any function over states
  - Successor function can also be anything

- Constraint satisfaction problems (CSPs):
  - A special subset of search problems
  - State is defined by variables X_i with values from a domain D (sometimes D depends on i)
  - Goal test is a set of constraints specifying allowable combinations of values for subsets of variables

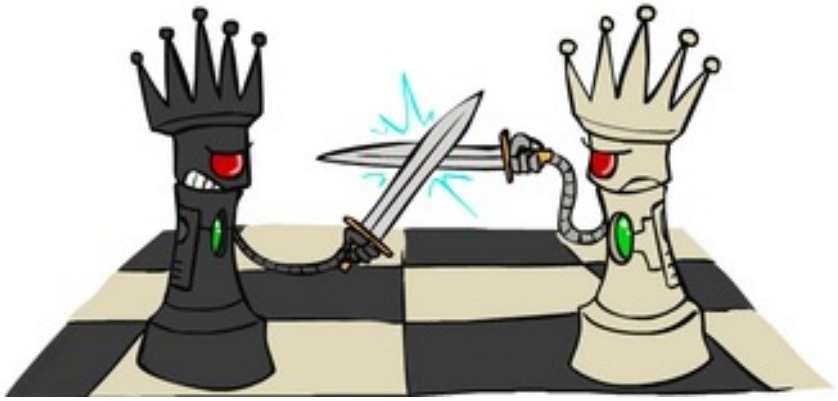
- Allows useful general-purpose algorithms with more power than standard search algorithms



Example: N-Queens

- Formulation 1:
  - Variables: X_{ij}
  - Domains: $\{0, 1\}$
  - Constraints:



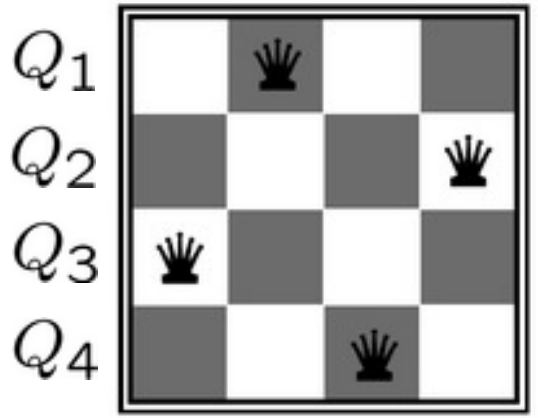


$$\forall i, j, k \quad (X_{ij}, X_{ik}) \in \{(0,0), (0,1), (1,0)\}$$
$$\forall i, j, k \quad (X_{ij}, X_{kj}) \in \{(0,0), (0,1), (1,0)\}$$
$$\forall i, j, k \quad (X_{ij}, X_{i+k,j+k}) \in \{(0,0), (0,1), (1,0)\}$$
$$\forall i, j, k \quad (X_{ij}, X_{i+k,j-k}) \in \{(0,0), (0,1), (1,0)\}$$

$$\sum_{i,j} X_{ij} = N$$

Example: N-Queens

- Formulation 2:
  - Variables: Q_k
  - Domains: $\{1, 2, 3, \ldots N\}$
  - Constraints:

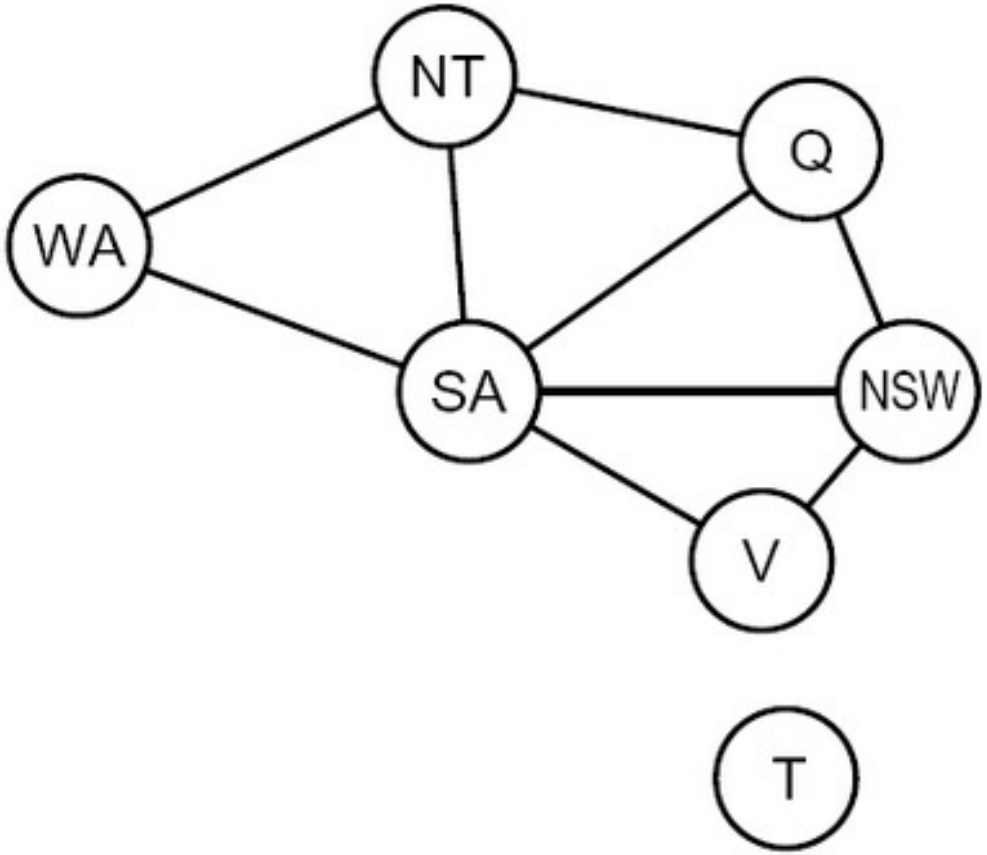


Implicit: $\forall i, j \;\; \text{non-threatening}(Q_i, Q_j)$

Explicit: $(Q_1, Q_2) \in \{(1,3), (1,4), \ldots\}$

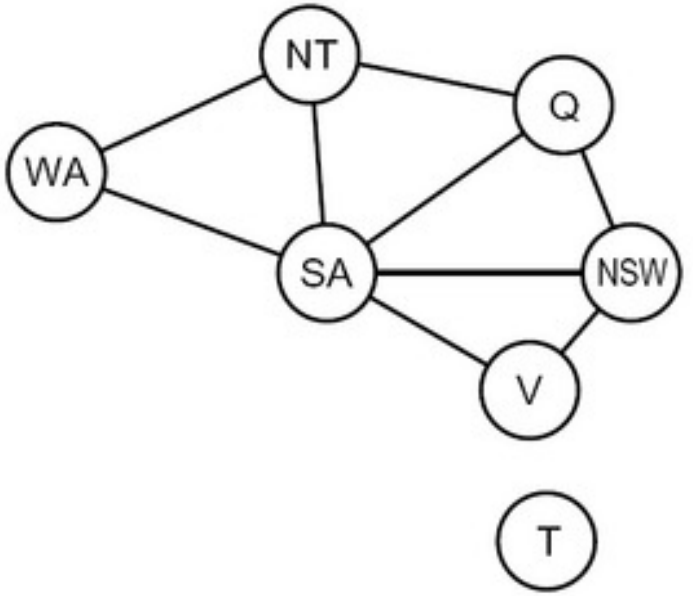
$\ldots$

Constraint Graphs



Constraint Graphs

- Binary CSP: each constraint relates (at most) two variables
- Binary constraint graph: nodes are variables, arcs show constraints
- Bonus: general-purpose CSP algorithms can use the graph structure to speed up search.
  E.g., Tasmania is an independent subproblem...



Example: Cryptarithmetic

- Variables:

  $F\ T\ U\ W\ R\ O\ X_1\ X_2\ X_3$

- Domains:

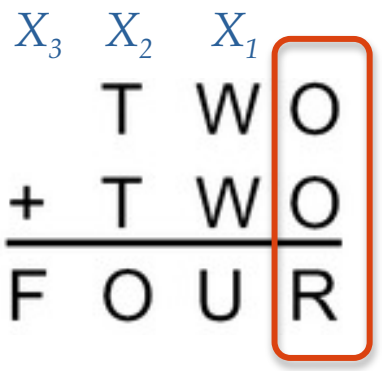
  $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

- Constraints:

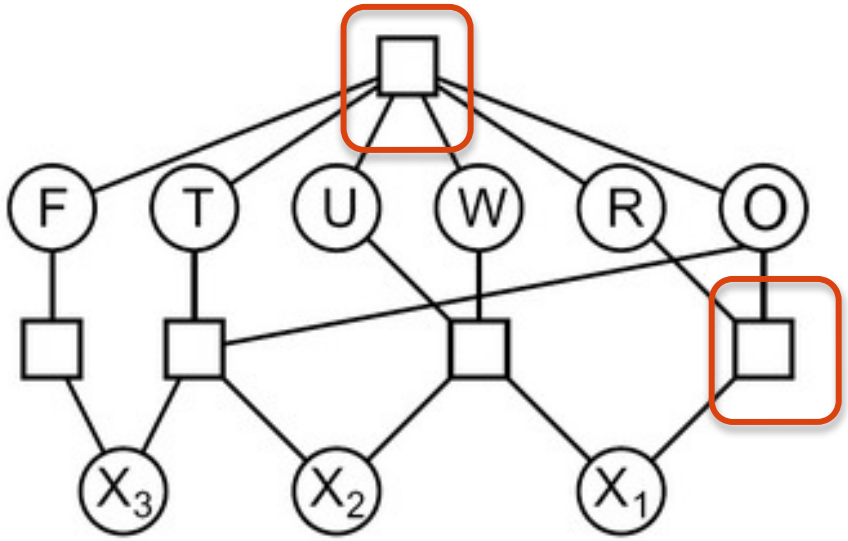
  $\text{alldiff}(F, T, U, W, R, O)$

  $O + O = R + 10 \cdot X_1$

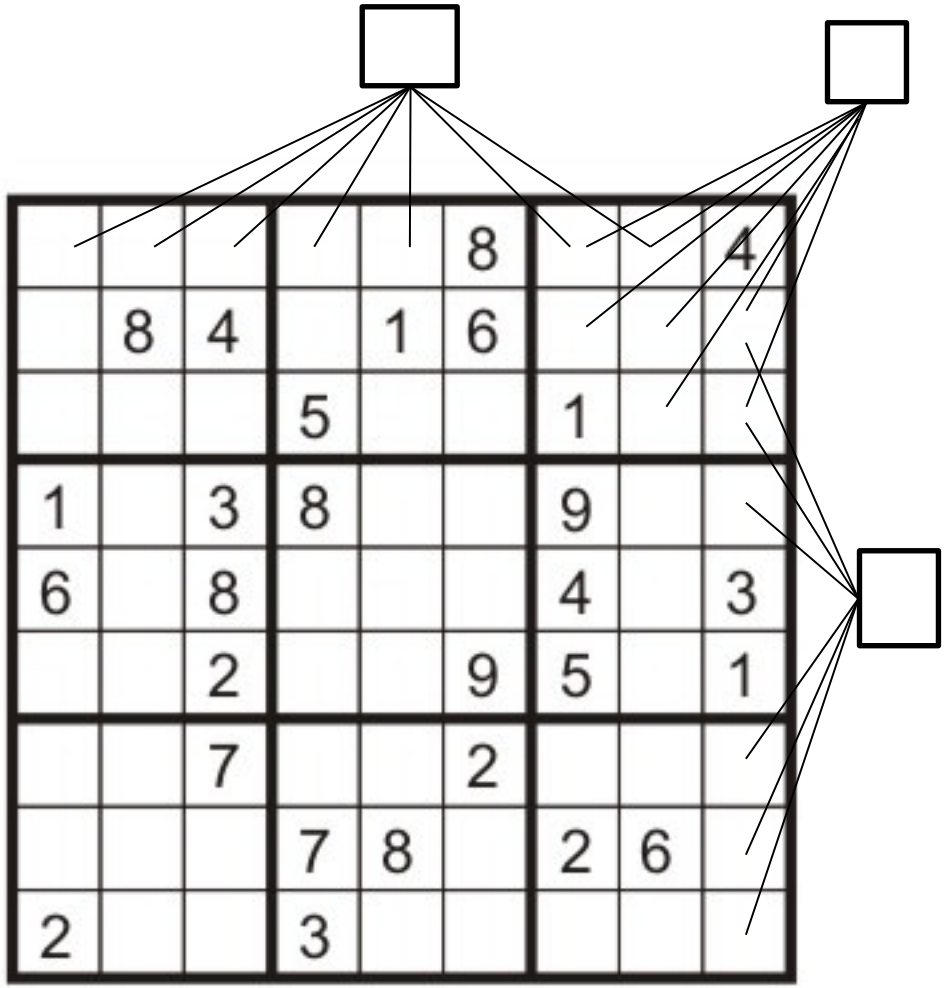
  . . .







Example: Sudoku



- Variables:
  - Each (open) square
- Domains:
  - {1,2,…,9}
- Constraints:

9-way alldiff for each column

9-way alldiff for each row

9-way alldiff for each block

(or can have a bunch of pairwise inequality constraints)

Varieties of CSPs and Constraints



Varieties of CSPs

- Discrete Variables
  - Finite domains
    - Size d domains with n variables means $O(d^n)$ complete assignments
    - E.g., Boolean CSPs, including Boolean satisfiability (NP-complete)
  - Infinite domains (integers, strings, etc.)
    - E.g., job scheduling, variables are start/end times for each job
    - Linear constraints solvable, nonlinear undecidable
- Continuous variables
  - E.g., start/end times for Hubble Telescope observations





Varieties of Constraints

- Varieties of Constraints
  - Unary constraints involve a single variable (equivalent to reducing domains), e.g.:

    $SA \neq green$

  - Binary constraints involve pairs of variables, e.g.:

    $SA \neq WA$

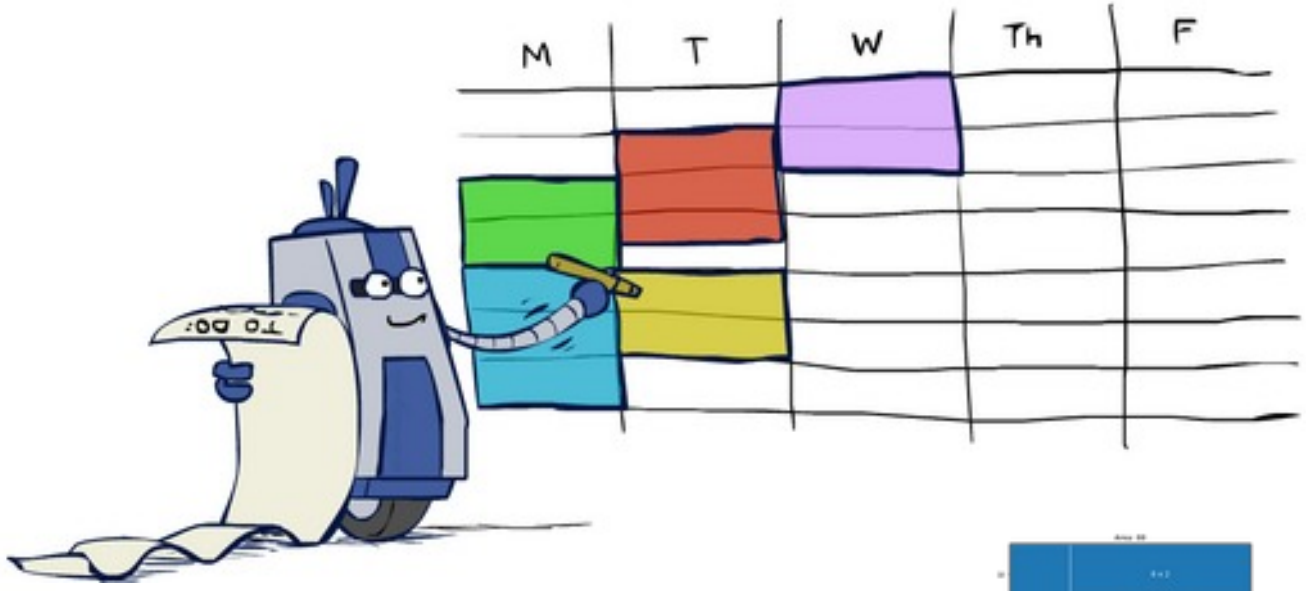
  - Higher-order constraints involve 3 or more variables:
    e.g., alldiff constraint, scheduling constraints



- Preferences (soft constraints):
  - E.g., red is better than green
  - Often representable by a cost for each variable assignment
  - Gives constrained optimization problems

Real-World CSPs

- Assignment problems: e.g., who teaches what class
- Timetabling problems: e.g., which class is offered when and where?
- Hardware configuration
- Transportation scheduling
- Factory scheduling
- Circuit layout
- Fault diagnosis
- … lots more!



- (demos at end of class)







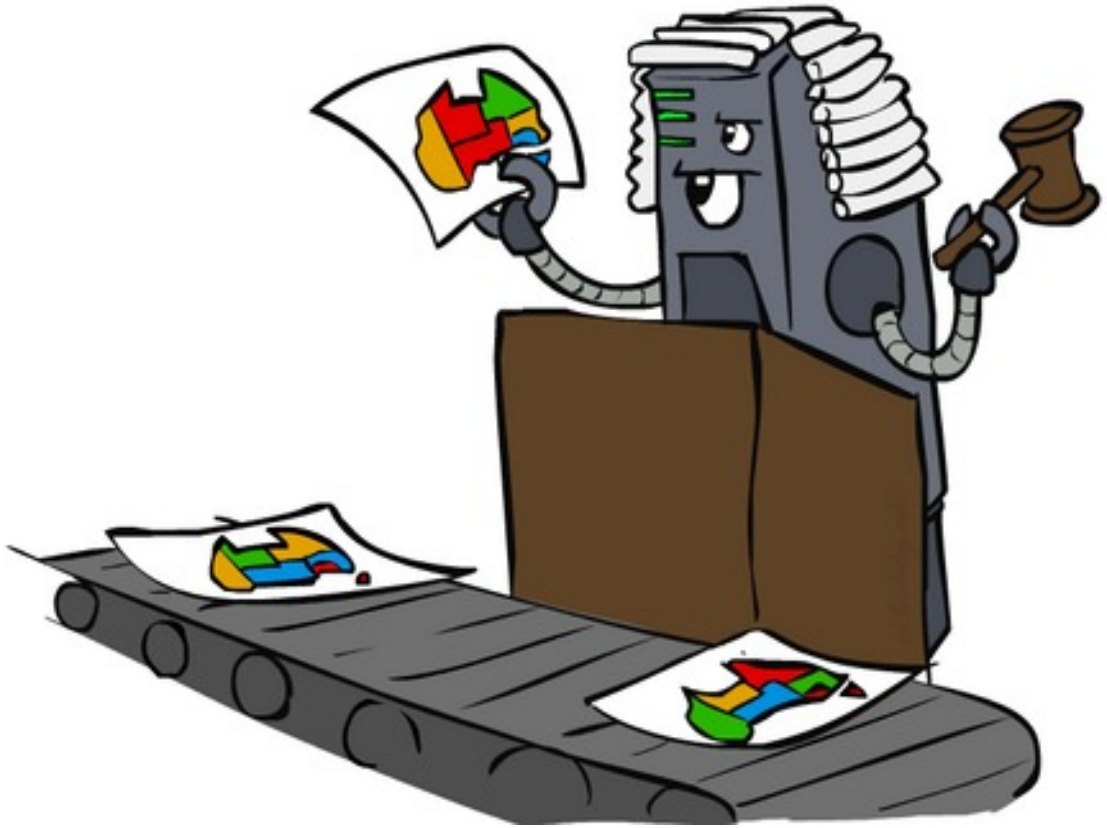


Solving CSPs



Standard Search Formulation

- Standard search formulation of CSPs
- States defined by the values assigned so far (partial assignments)
  - Initial state: the empty assignment, {}
  - Successor function: assign a value to an unassigned variable
  - Goal test: the current assignment is complete and satisfies all constraints
- We'll start with the straightforward, naïve approach, then improve it



Search Methods

- What would BFS do?

{}

{WA=g} {WA=r} … {NT=g} …



Search Methods

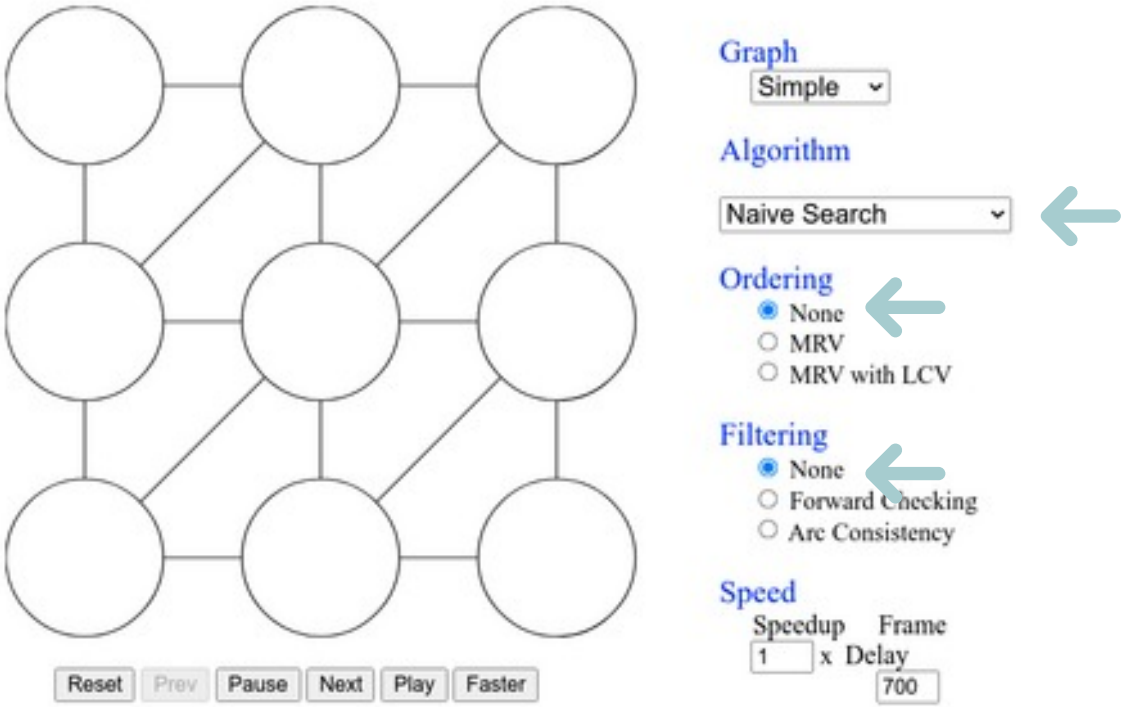
- What would BFS do?

- What would DFS do?
  - let's see!

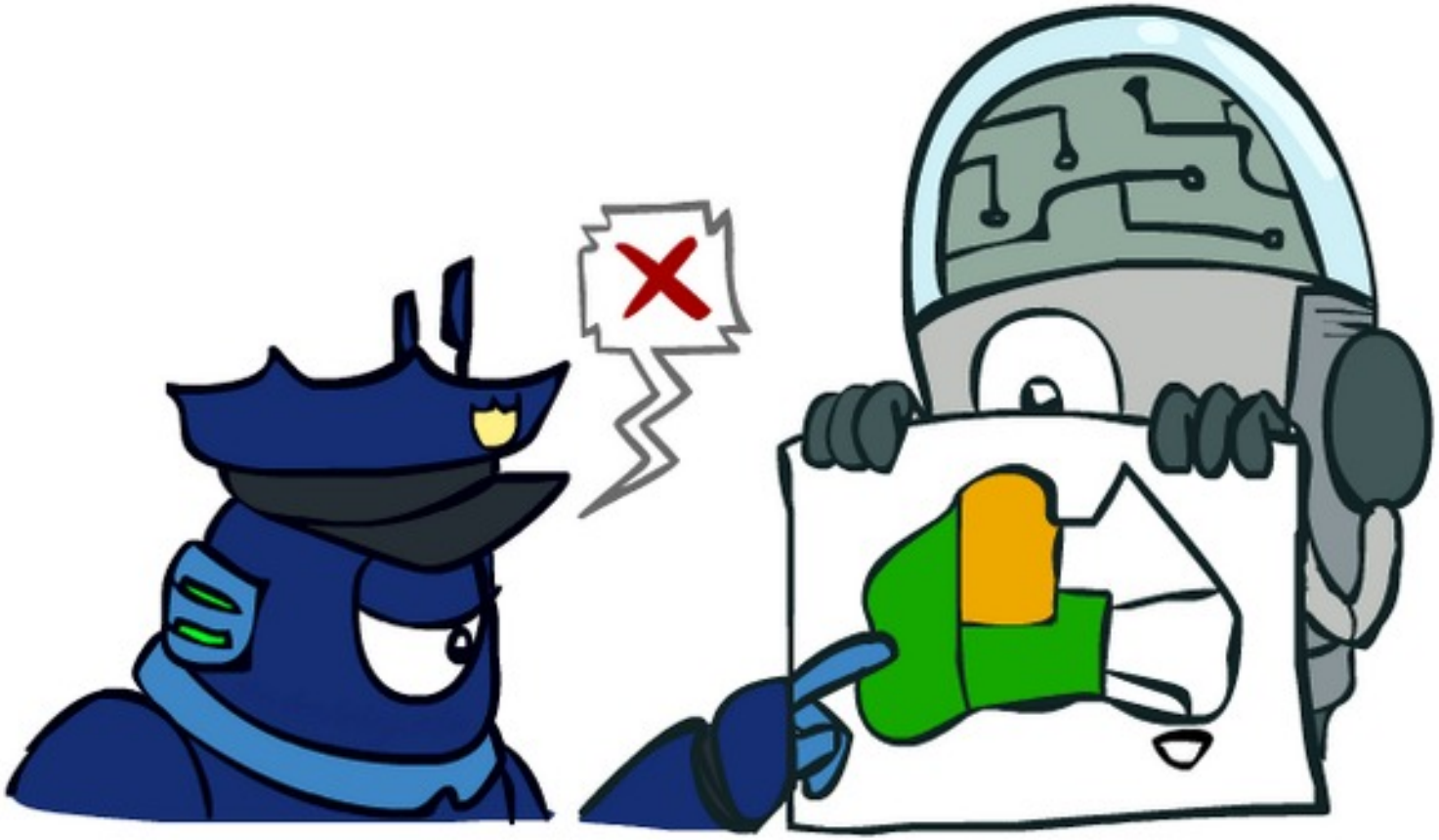


Demo

https://www.cs.cmu.edu/~15281/demos/csp_backtracking/

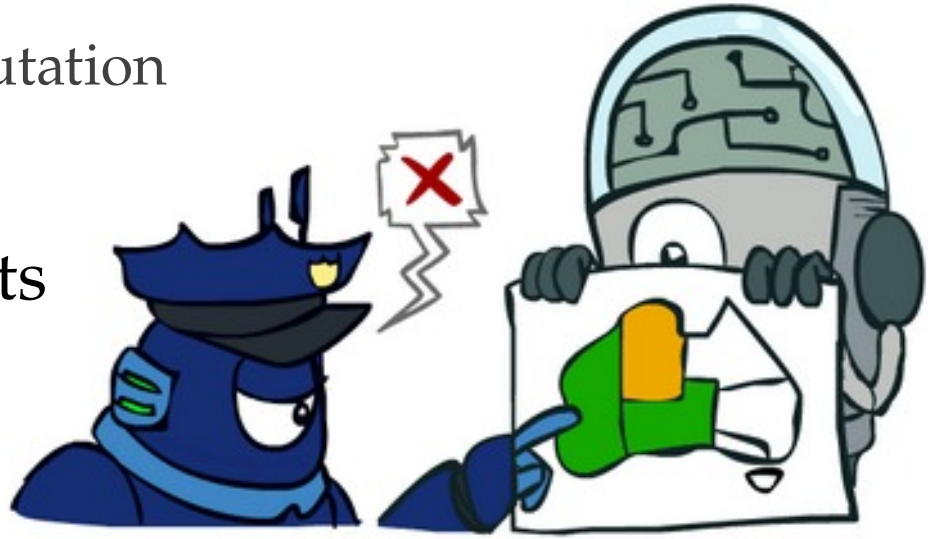


Backtracking Search

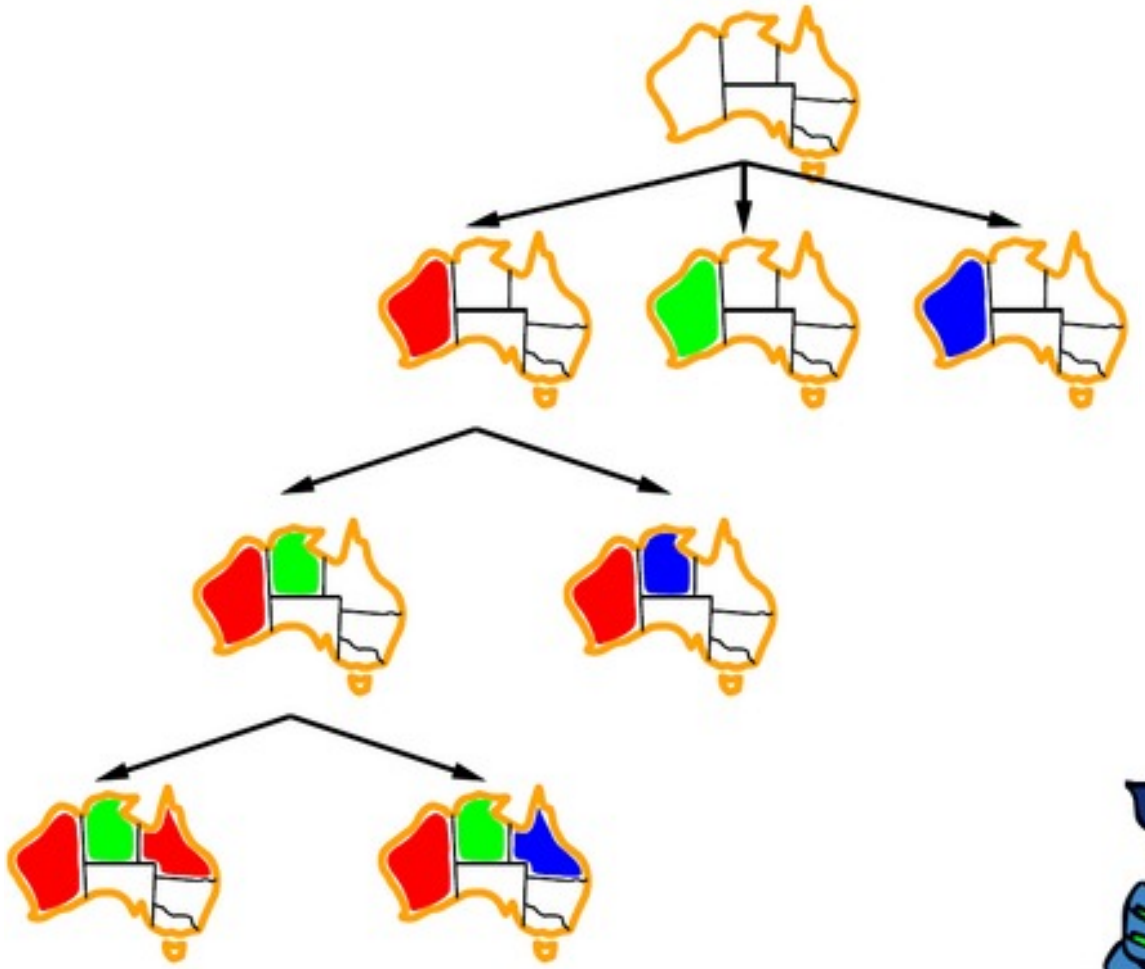


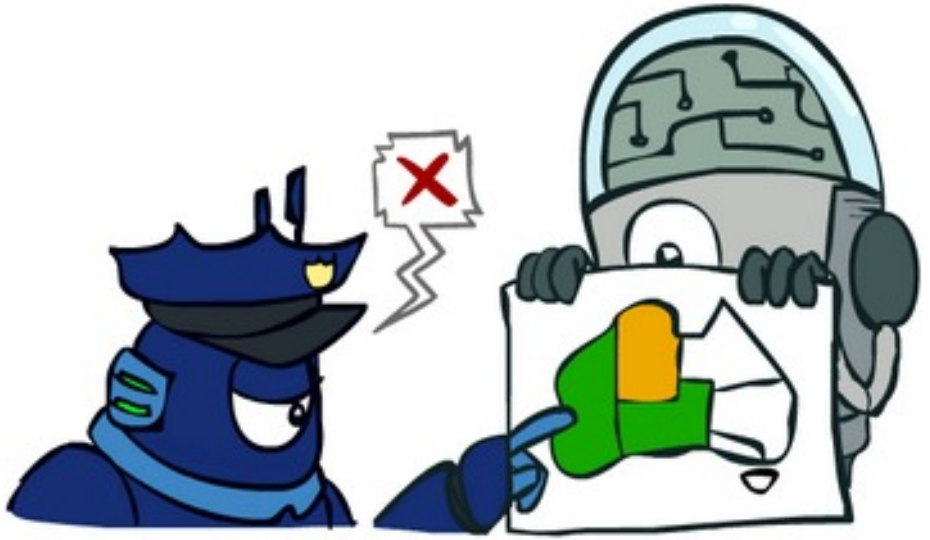
Backtracking Search

- Idea 1: Consider only one variable in every search node
  - Variable assignments are commutative, so fix ordering -> better branching factor!
  - I.e., [WA = red then NT = green] same as [NT = green then WA = red]
  - Only need to consider assignments to a single variable at each level of the search tree
- Idea 2: Check constraints as you go
  - I.e. ignore values that are not consistent with already assigned variables
  - "Incremental goal test"
  - Checking each constraint might require some computation
- Depth-first search with these two improvements is called *backtracking search*



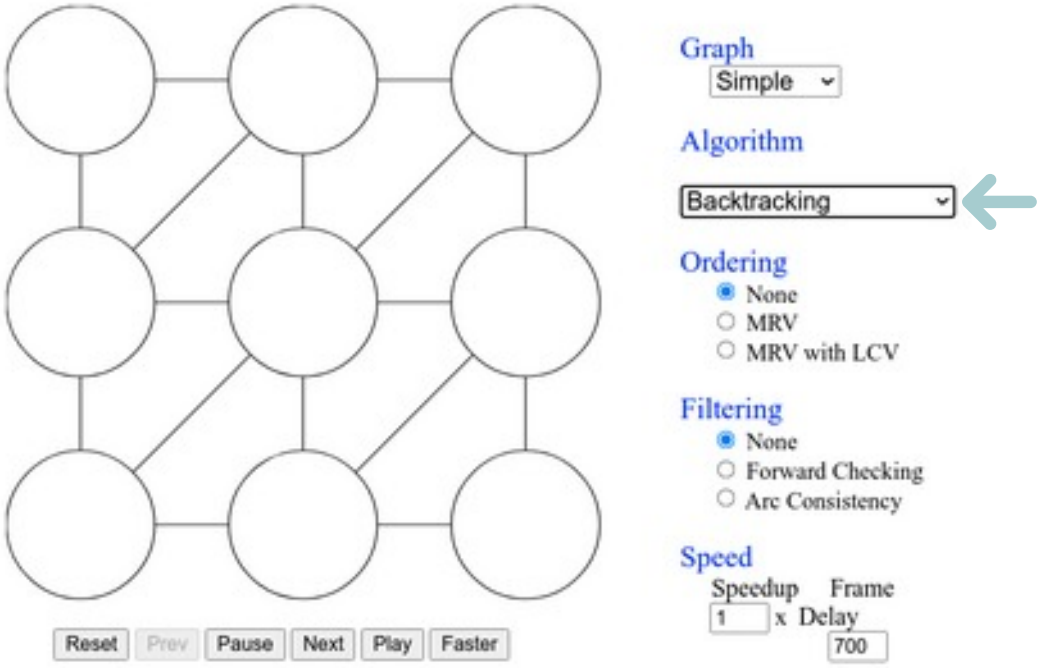
Backtracking Example





Demo

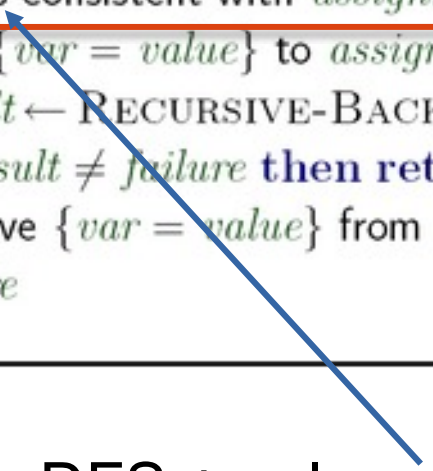
https://www.cs.cmu.edu/~./15281/demos/csp_backtracking/



Backtracking Search

```
function BACKTRACKING-SEARCH(csp) returns solution/failure
    return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
    if assignment is complete then return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
        if value is consistent with assignment given CONSTRAINTS[csp] then
            add {var = value} to assignment
            result ← RECURSIVE-BACKTRACKING(assignment, csp)
            if result ≠ failure then return result
            remove {var = value} from assignment
    return failure
```



- Backtracking = DFS + value-consistency + fail-on-violation
- What are the remaining agorithm design choices?

Improving Backtracking

- General-purpose ideas give huge gains in speed
- Ordering:
  - Which variable should be assigned next?
  - In what order should its values be tried?
- Filtering: Can we detect inevitable failure early?



Filtering



Keep track of domains for unassigned variables and cross off bad options

Filtering: Forward Checking

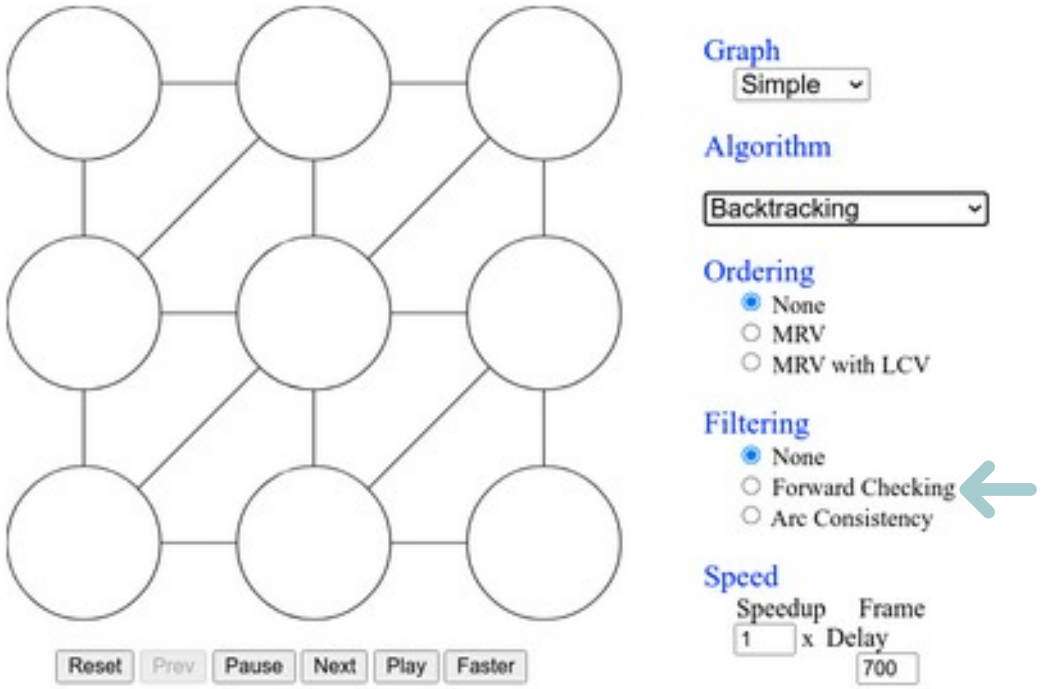
- Filtering:
  - Will need to keep track of domains for unassigned variables, to filter out bad options
- Forward checking:
  - Cross off values that violate a constraint when added to the existing assignment





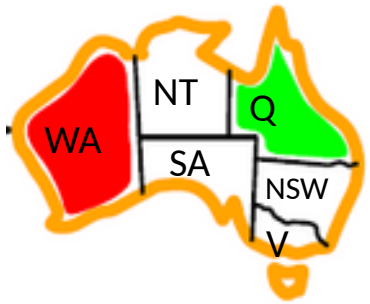
Demo

https://www.cs.cmu.edu/~./15281/demos/csp_backtracking/



Filtering: Constraint Propagation

- Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures:

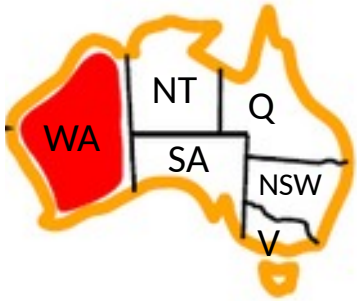


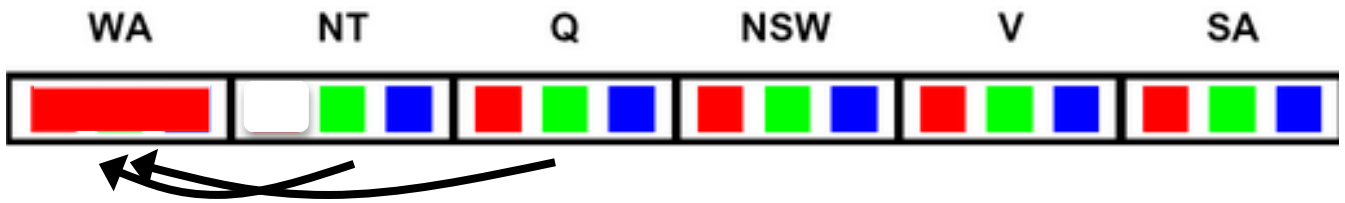


- NT and SA cannot both be blue!
- Why didn’t we detect this yet?
- Need *constraint propagation:* reason about consistency of each constraint

Consistency of A Single Arc

- An arc X → Y is consistent iff for *every* x in the tail there is *some* y in the head which could be assigned without violating a constraint





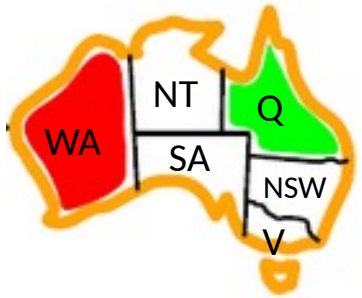


Delete from the tail!

Forward checking: Enforcing consistency of arcs pointing to each new assignment

Arc Consistency of an Entire CSP

- Constraint propagation: arc consistency to make sure all arcs are consistent:





- Important: If X loses a value, neighbors of X need to be rechecked!
- Arc consistency detects failure earlier than forward checking
- Can be run as a preprocessor or after each assignment

Remember: Delete from the tail!

Enforcing Arc Consistency in a CSP

```
function AC-3( csp) returns the CSP, possibly with reduced domains
  inputs: csp, a binary CSP with variables {X_1, X_2, ..., X_n}
  local variables: queue, a queue of arcs, initially all the arcs in csp

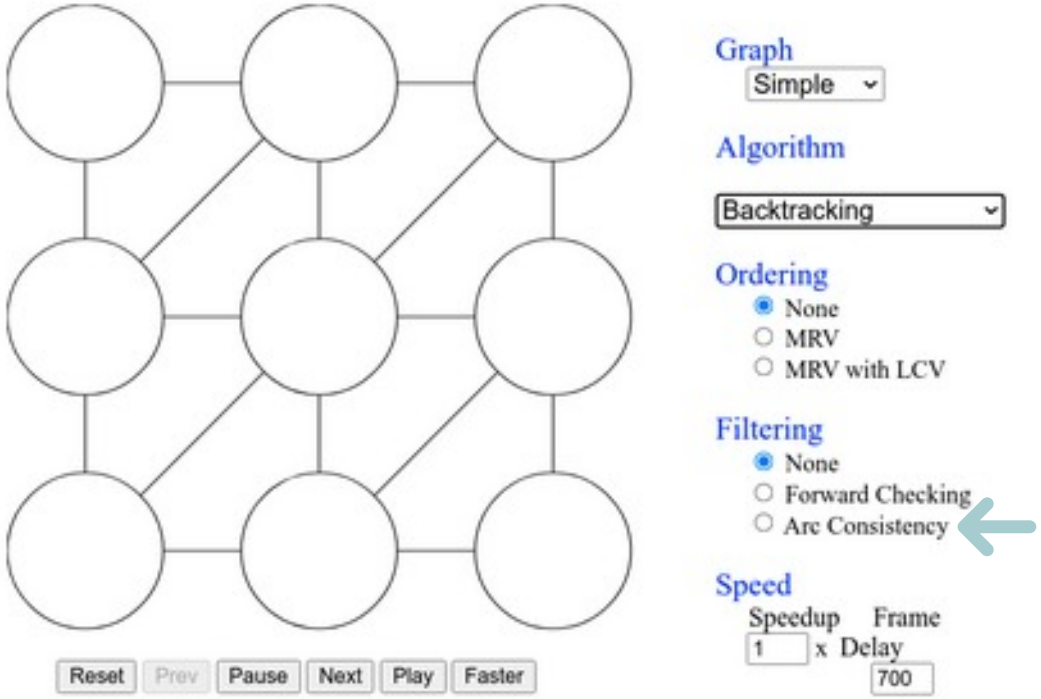
  while queue is not empty do
    (X_i, X_j) ← REMOVE-FIRST(queue)
    if REMOVE-INCONSISTENT-VALUES(X_i, X_j) then
      for each X_k in NEIGHBORS[X_i] do
        add (X_k, X_i) to queue

function REMOVE-INCONSISTENT-VALUES( X_i, X_j) returns true iff succeeds
  removed ← false
  for each x in DOMAIN[X_i] do
    if no value y in DOMAIN[X_j] allows (x,y) to satisfy the constraint X_i ↔ X_j
      then delete x from DOMAIN[X_i]; removed ← true
  return removed
```

- Runtime: $O(n^2d^3)$, can be reduced to $O(n^2d^2)$

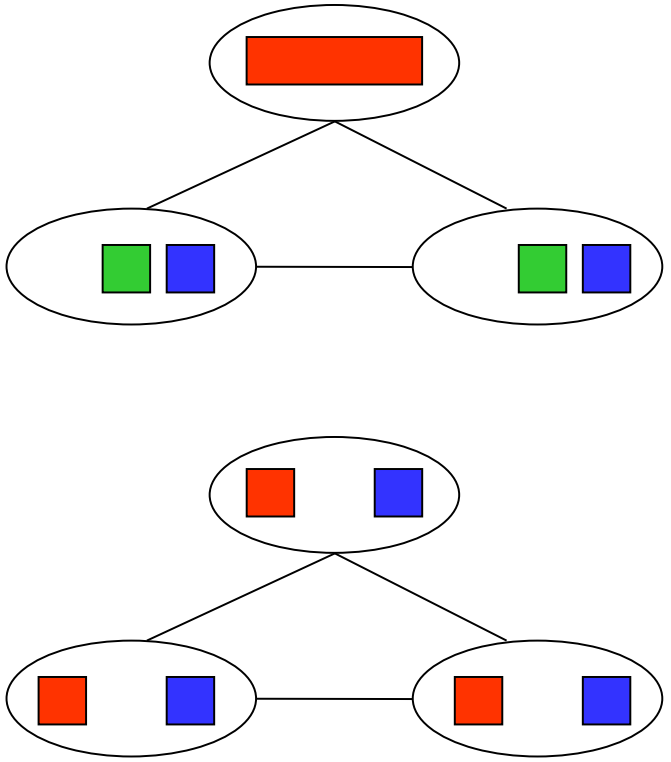
Demo

https://www.cs.cmu.edu/~./15281/demos/csp_backtracking/



Limitations of Arc Consistency

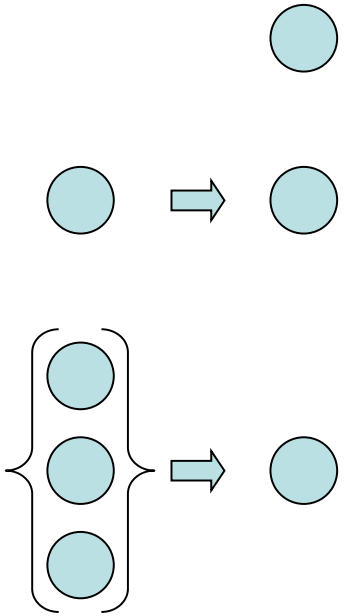
- After enforcing arc consistency:
  - Can have one solution left
  - Can have multiple solutions left
  - Can have no solutions left (and not know it)
- Arc consistency has to run inside a backtracking search!



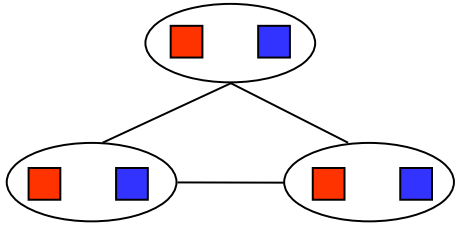
What went wrong here?

K-Consistency

- Increasing degrees of consistency
  - 1-Consistency (Node Consistency): Each single node's domain has a value which meets that node's unary constraints
  - 2-Consistency (Arc Consistency): For each pair of nodes, any consistent assignment to one can be extended to the other
  - K-Consistency: For each k nodes, any consistent assignment to k-1 can be extended to the kth node.



- Higher k is more expensive to compute



Strong K-Consistency

- Strong k-consistency: also k-1, k-2, … 1 consistent
- Claim: strong n-consistency means we can solve without backtracking!
- Why?
  - Choose any assignment to any variable
  - Choose a new variable
  - By 2-consistency, there is a choice consistent with the first
  - Choose a new variable
  - By 3-consistency, there is a choice consistent with the first 2
  - …
- Lots of middle ground between arc consistency and n-consistency! (e.g. k=3, called path consistency)
- Modern solvers use arc consistency + smart search

Ordering

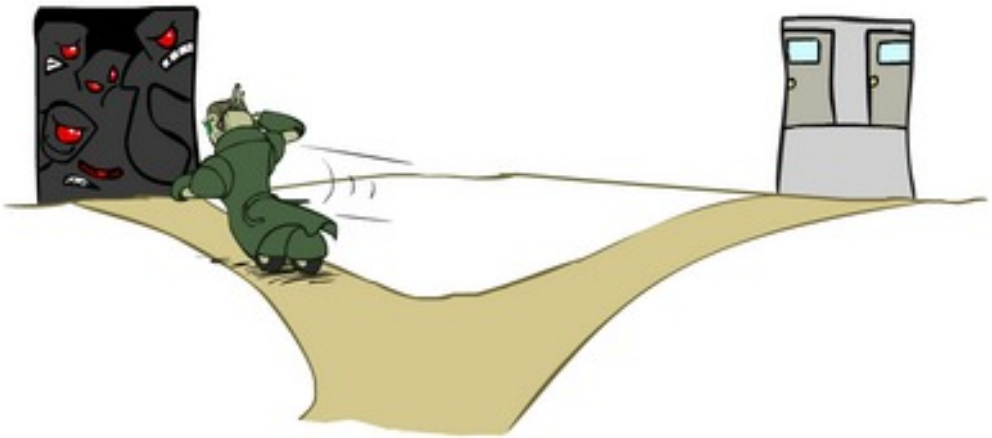


Ordering: Minimum Remaining Values

- Variable Ordering: Minimum remaining values (MRV):
  - Choose the variable with the fewest legal values left in its domain

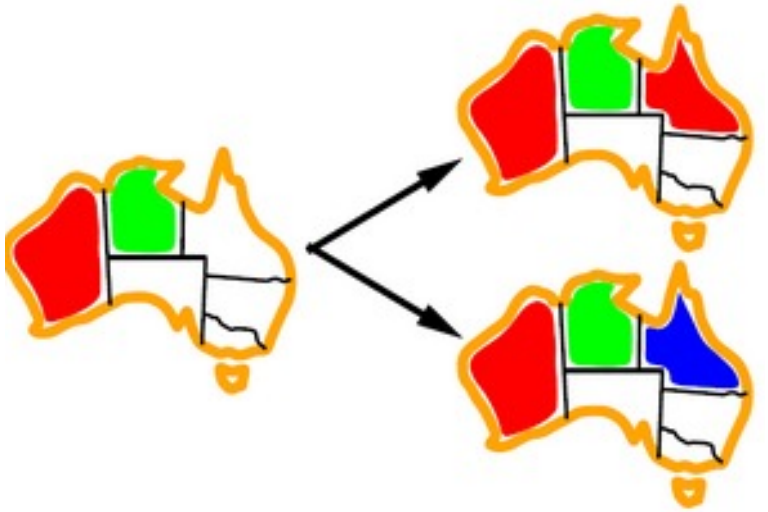


- Why min rather than max?
- "Fail-fast" ordering
- Also called "most constrained variable"



Ordering: Least Constraining Value

- Value Ordering: Least Constraining Value
  - Given a choice of variable, choose the *least constraining value*
  - I.e., the one that rules out the fewest values in the remaining variables
  - Note that it may take some computation to determine this! (E.g., rerunning filtering)



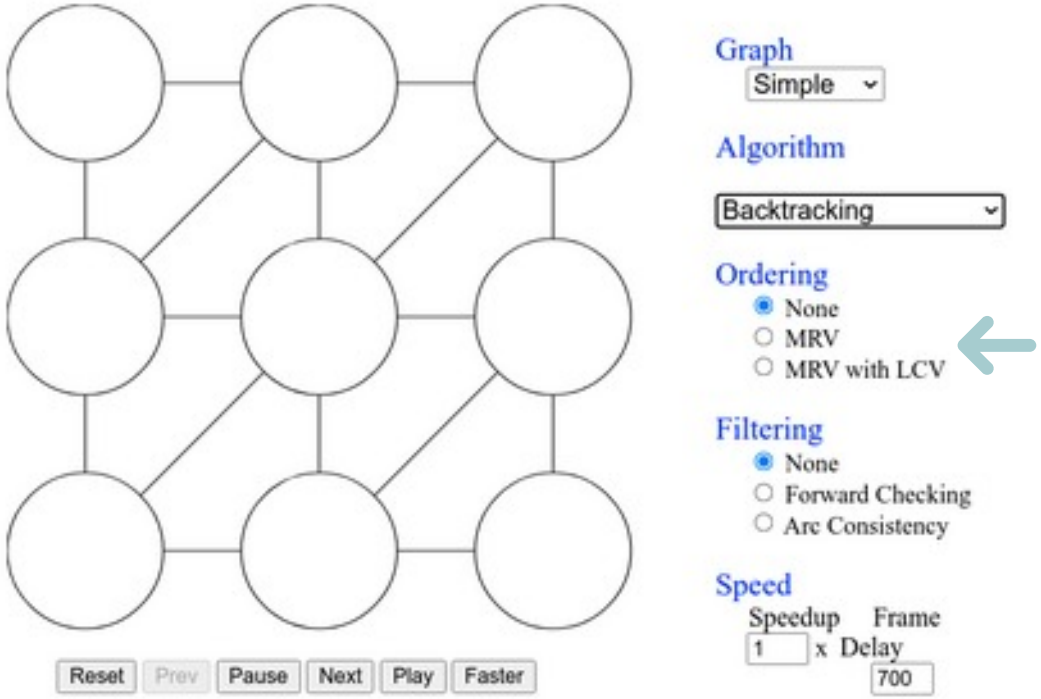
- Why least rather than most?

- Can combine different variable and value orderings



Demo

https://www.cs.cmu.edu/~./15281/demos/csp_backtracking/



Value Symmetry

- In many CSP problems there are symmetries
- For instance, in map-coloring with 3 colors, there are 3! = 6 symmetric solutions.
  These can be obtained by simply permuting the colors.
- For instance
  - WA = red, NT = green, SA = blue, Q = red, ...
  - WA = red, NT = blue, SA = green, Q = red, ...
  - WA = blue, NT = green, SA = red, Q = blue, ...
  - …
- We can reduce the search space by using symmetry breaking constraints
- WA < NT, WA < SA, NT < SA, ... (thus WA < NT < SA < ...)



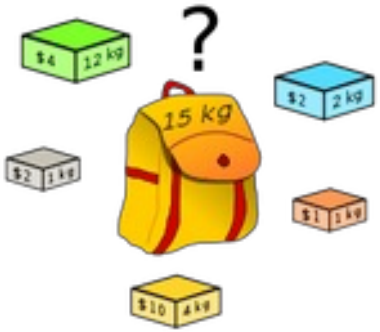
Constraint Optimisation?

- Most real-world problems involve an optimisation problem:

- Variables (with a finite domain)
- Constraints
- Objective function

COP = Constrained Optimisation Problem

Modeling



Knapsack:

Model =

- Variables, with a domain
- Constraints over variables
- Optionally: an objective

- gr, bl, og, ye, gy :: {0,1}
- 12*gr + 2*bl + 1*og + 4*ye + 1*gy <= 15
- maximize(4*gr + 2*bl + 1*og + 10*ye + 2*gy)

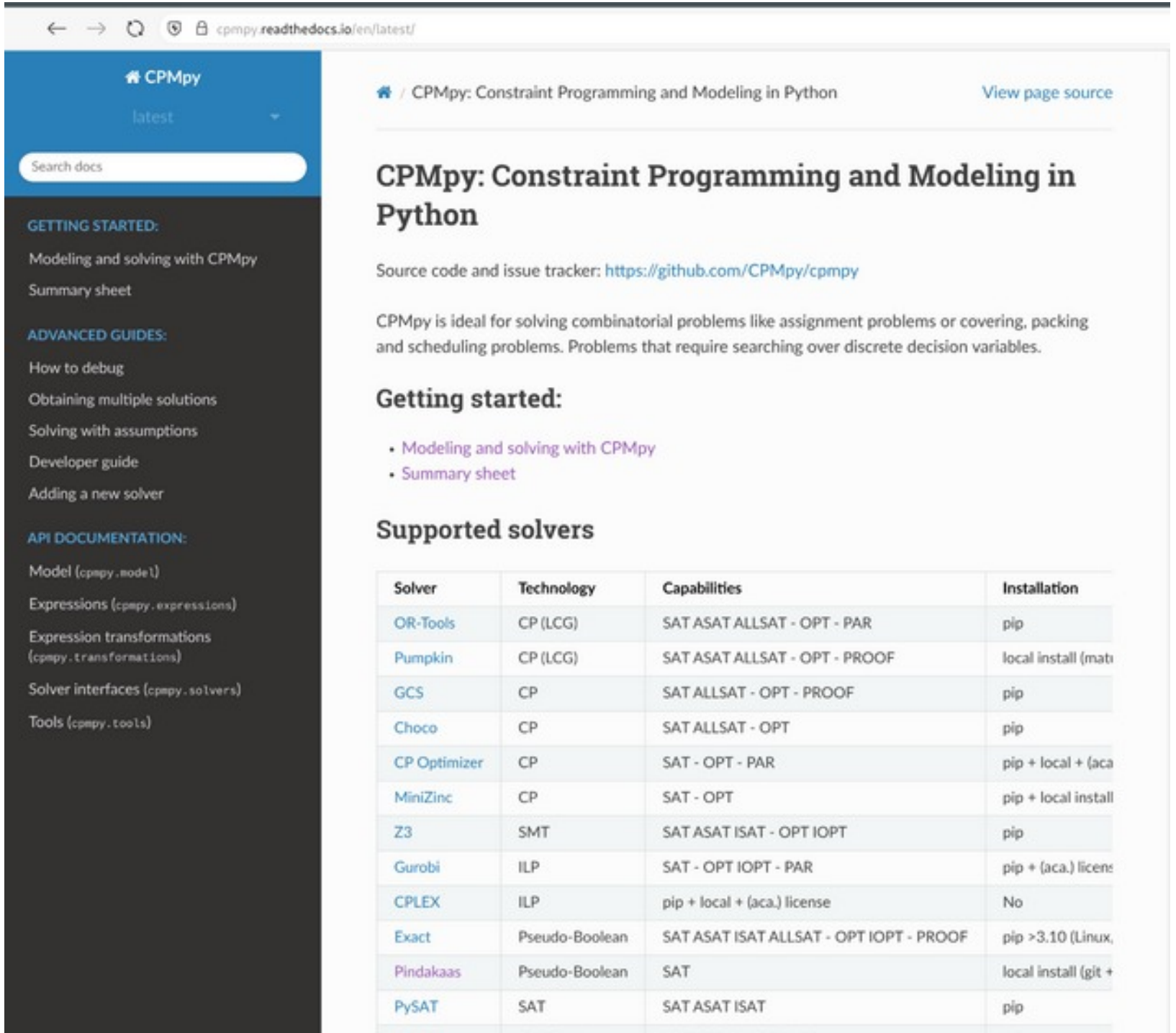
Model.solve()

Optimisation: branch-and-bound

Key idea: maintain the 'best' objective value so far

constrain solutions to be better than the 'best'

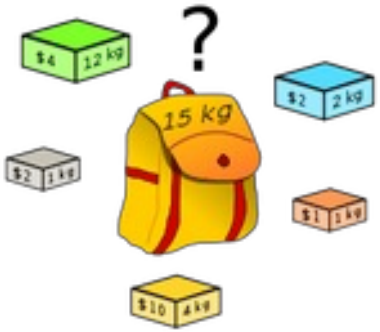
0. set bound = -infinity (maximization problem)
   Add a constraint: *objective_function()* > *bound*
1. Do backtracking search,
   ensuring all constraints are satisfied (incl objective function constraint)
2. If a solution is found: compute its objective function value v
   update bound to 'v' in the objective constraint
3. continue (with updated objective function constraint)



https://cpmpy.readthedocs.io/

(seen extensively in the course “Constraint Solving”)

Modeling



Knapsack:

Model =

- Variables, with a domain
- Constraints over variables
- Optionally: an objective

```
model = Model()

gr,bl,og,ye,gy = boolvar(shape=5)

model += (12*gr + 2*bl + 1*og + 4*ye + 1*gy <= 15)

model.maximize(4*gr + 2*bl + 1*og + 10*ye + 2*gy)


model.solve()
```

Model.solve()

```
print(gr.value(), bl.value(), og.value(), ye.value(), gy.value())
```

```
0 1 1 1 1
```

Integrated constraint solving

- Many integrations of constraint solving with machine learning, an example:





https://visualsudoku.cs.kuleuven.be

GET IT ON Google Play

CPMpy demo

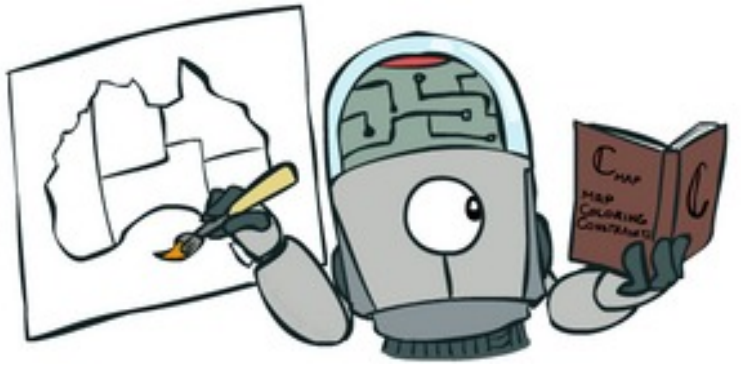
- A modern C(S)P library https://cpmpy.readthedocs.io/

- Graph coloring Australia example:
- https://github.com/CPMpy/cpmpy/blob/master/examples/map_coloring_oz.ipynb
- N-queens up to N=1000 example:
- https://github.com/CPMpy/cpmpy/blob/master/examples/nqueens_1000.ipynb
- Industrial examples with nice visualisations (on toy data):
- https://github.com/CPMpy/cpmpy/tree/master/examples/






Summary



CSPs are a special kind of search problem:

- States are partial assignments
- Goal test defined by constraints

Basic solution: backtracking search

Speed-ups:

- Filtering
- Ordering
- (Structure)
- (Symmetry breaking)

Numerous applications